Bridge structural health monitoring dashboard

SpanGuard: project handoff

github.com/praveenregar30/spanguard | prepared 10 November 2026

Status as of 10 November 2026. This file tells a person or another AI assistant everything needed to continue the work: what exists, where it is, how it fits together, what has been tested, and what is still open.

1. What the project is

A web dashboard for a road-bridge structural health monitoring (SHM) system.

- **Hardware:** 3 sensor nodes (ESP32 + ADXL345 accelerometer) on the bridge deck at 1/4, 1/2 and 3/4 of the span.
- **Base station:** a Raspberry Pi. Every 5 minutes it records a 30-second capture (200 Hz, 3 axes per node) and scores it with an autoencoder (AANN).
- **Score:** a ratio where 1.0 is the alarm level. ALARM means at least 3 of the last 5 captures were above 1.0; WARNING means at least one was.
- **Transport:** the Pi sends results over a 4G SIM to an EMQX Cloud Serverless MQTT broker. The dashboard connects to that broker over WebSockets: `wss://<address>:8084/mqtt`, with its own username and password.
- **History:** a laptop receiver stores all messages in `remote.sqlite`, which the dashboard can open with a file picker.

The owner is a student at IIT Mandi (GitHub `praveenregar30`, repository name `spanguard`).

2. Where everything is

Location	What it is
`C:\Users\77263\bridge-shm\`	Project folder and git repository (branch `main`).
`index.html`	**The deliverable.** The whole dashboard in one file (HTML + CSS + JS, about 170 kB). It is generated: never edit it by hand.
`src\01-head.html`	`<head>`, all CSS (colour tokens, light and dark themes, responsive rules) and all page markup for the 6 pages. Ends with an open `<script>` tag.
`src\02-core.js`	Helpers, the data store `S`, healthy baseline, node status, preview decoding, FFT and Welch spectrum, chart components (`lineChart`, `waveStack`), icons.
`src\03-overview.js`	Overview page: animated bridge replay (`R` object), status headline, instrument strip, 24 h score chart, node list, event timeline.
`src\04-pages.js`	Vibration page (`VIB`), raw `.npz` reader (`RAW`, `unzip`, `parseNpy`, `parseRaw`), Trends (`TR`), Events (`EV`), System.
`src\05-app.js`	Remote commands, MQTT connection (`MQ`), SQLite loading, demo simulator (`DEMO`), router, theme, event wiring, `init()`.

Location	What it is
`src\build.sh`	Concatenates the 5 source parts into `index.html`. Run `sh src/build.sh` after every edit.
`broker_test.py`	Fake Raspberry Pi for testing (Python, needs only `paho-mqtt`). Publishes every message type in the exact formats and answers commands.
`tools\cdp-test.mjs`	Headless Chrome test driver (Node 22+, no packages). Runs a JSON list of steps: navigate, run JS in the page, screenshot, report console errors.
`tools\example-steps.json`	Example test: triggers the alarm in demo mode and checks the phone layout.
`README.md`	Short public description and setup steps for GitHub visitors.
`HANDOFF.md`	This file.
`.gitignore`	Ignores `__pycache__`, `*.sqlite`, `*.npz`.

Published demo (private claude.ai page): https://claude.ai/artifact/FKQX6z9kBuDdMEZdQjQWmR. It is published from a variant of `index.html` with the outer `<!doctype>`, `<html>`, `<head>` and `<body>` tags stripped, because the artifact host adds its own. To update it, rebuild, strip those tags again, and republish to the same URL.

GitHub: not pushed yet. The owner will push to `https://github.com/praveenregar30/spanguard` (steps in section 9). GitHub Pages would then serve the dashboard at `https://praveenregar30.github.io/spanguard/`.

3. Rules from the original brief (keep these)

- **One self-contained HTML file, no build tools for the user.** It must work from `file://` in Chrome and on any static host.
- **Libraries:** only mqtt.js `5.10.1` and sql.js `1.10.3` (the asm.js build `sql-asm.js`, so no `.wasm` fetch), both from cdn.jsdelivr.net and loaded lazily (only when connecting or opening a file), so demo mode works offline.
- **Fonts:** Barlow (body) and Barlow Semi Condensed (headlines and numbers) from Google Fonts, with system fallbacks.
- **Palette:** survey paper and instruments. Light: background `#edf1ec`, panels `#f6f8f5`, rules `#cbd5cd`, ink `#1d2b35`, Prussian-blue traces `#1d5c8f`. Dark: a night instrument panel (`#10171c`, pale blue traces). Green, amber and red are used only for status. Dark mode follows the system setting, with a manual toggle.
- **Style:** flat panels, 1px rules, 8px radius, no drop shadows, no all-caps labels, no "A · B · C" text strings, sentence case, plain English ("alarm level", not "threshold"). Charts sit on a faint graph-paper grid, with tooltips and a legend when there are 2 or more series.
- **Responsive** down to 390 px: larger bridge markers on phones, tabs that scroll sideways, visible keyboard focus. Respect `prefers-reduced-motion`: no autoplay, show the strongest moment instead.
- **Hash routing:** `#overview`, `#vibration` (or `#vibration/<capture_id>`), `#trends`, `#events`, `#system`, `#connect`.

4. Message formats (all under `shm/<site>/`, default site `bridge01`)

- **capture** (one per capture): `{site, id, ts (unix s), mode ("monitor"|"collect"), state ("NORMAL"|"WARNING"|"ALARM"), q (quality ok), flags [strings], score, ratio, thr, why, temp (°C), freq [3 Hz], mac [3, 0 to 1], nodes {"n1": ratio or null if bad data}}`. A node missing from `nodes` means it was silent.
- **preview** (one per capture, about 9 to 14 kB): `{site, id, ts, state, mode, q, ratio, why, fs: 50, src_fs: 200, dur: 30, df, events [[start_s, end_s]], live, nodes: [{node, ok, rms, peak, noise, n, scale, coding, wave, spec, spikes [[t, mg]], glitches [[t, mg]]}]}`.
- `wave`: base64, then zlib inflate (`DecompressionStream("deflate")`), then little-endian int16, then a cumulative sum if `coding` is `"delta"`, then multiply by `scale`. The result is vertical acceleration in mg at 50 Hz, gravity removed.
- `spec`: integers equal to `round(100 * log10(PSD))`, so dB = value / 10, and bin k sits at `k * df` Hz (0 to 25 Hz).
- **event:** `{type: "state", from, to, capture_id, ts, ratio, why}` or `{type: "fault" | "recovered", what, ts}`.
- **state** (retained): `{state, since, why}`.
- **online** (retained): `"1"` or `"0"`. `"0"` is also the Pi's last-will message.
- **heartbeat** (every 15 min, not retained, has no `ts`; the dashboard stamps it with the time it arrives): `{uptime_s, cpu_temp_c, disk_free_gb, backlog, modem {signal_pct, tech, operator}, nodes [{node_id, age_s}], faults [strings], live_until}`.
- **cmd** (published by the dashboard): `{cmd, token, ...}` where `cmd` is `ping`, `status`, `sms_test`, `set_mode {mode}`, `send_raw {capture_id}`, `live_view {minutes ≤ 30}` or `live_stop`.
- **cmd_reply:** `{cmd, ok, detail, live_until?, error?}`.
- **raw/** (binary `.npz`): a numpy zip with `x` (channels × samples; int16 counts with `lsb_g` in g per count, or float32 in g), `channel_names` (such as `"n1x"`), `fs`, `ts` and `capture_id`. The reader handles zip64 extra fields and `.npy` versions 1 to 3, and inflates with `DecompressionStream("deflate-raw")`.

History file `remote.sqlite` **tables:** `captures(site, capture_id, ts, json)`, `events(site, ts, json)`, `heartbeats(site, ts, json)`, `status(site, key, value, ts)` with keys `state` and `online`, and `previews(site, capture_id, ts, json)`.

5. How the code works

- **Store** `S` (`02-core.js`): `caps` (sorted by ts) plus `capById`, `prevs` (Map of previews), `events`, `hbs` (heartbeats), `piState`, `online`, `raws` (open `.npz` files), `liveUntil`, `file`, and `ver`. `ver` increases on every change and is used to invalidate caches.
- **Data flows in** through `addCapture`, `addPreview`, `addEvent` and `addHeartbeat` from three sources: MQTT (`onMqttMessage`), SQLite (`openSqlite`) and the demo (`demoTick`). Then `scheduleRender()` re-renders the current page, throttled to one render per 200 ms.
- **Healthy baseline** (`baseline()`): the median freq and MAC of the first 50 captures with `q` true and state NORMAL. It needs at least 5. Frequency changes below −3 % are shown in red.
- **Node status** (`nodeStatuses()`): silent if not heard for more than 10 minutes (from captures and heartbeat `age_s`), bad data if the latest capture has `null` for that node, otherwise reporting.

- **"Now":** `nowRef()` returns real time when live (demo or MQTT) and the latest capture time when only a history file is open, so a history file doesn't show every node as silent.
- **Bridge replay** (`03-overview.js`): an SVG side view with viewBox `0 76 1000 254`, the deck from x=80 to x=920, deck top at y=190. Each frame, the 3 node values are fitted exactly by the first 3 sine modes, `sin(kπx)` for k=1 to 3, using the precomputed inverse matrix `MINV`. That gives a smooth curve that is zero at the abutments. A car drives across during each vehicle-crossing event. A newer capture waits in `R.pending` until the current replay ends, or a "Show the newer capture" button appears if the user paused.
- **Charts:** `lineChart(el, opts)` (time or numeric x, reference lines, vertical lines, bands, gaps, hover tooltip, click) and `waveStack(el, opts)` (stacked waveforms on a shared scale, min/max decimated, spikes as red dots, bad reads as hollow squares, vehicle crossings shaded, hover crosshair).
- **Spectra:** previews carry their own spectrum. The "typical" curve is the per-bin median of the last 15 normal previews (it needs at least 5). Raw files get a Hann-window Welch spectrum with 5.12 s segments, computed in the browser (`welch()`).
- **MQTT** (`mqttConnect`): normalises the address (accepts a bare host, builds `wss://host:8084/mqtt`, fixes a doubled `.emqxsl.com`). It sets `reconnectPeriod: 0` until the first successful connect, then 5 s. Its own 16 s timer reports a connack timeout. `explainMqttError` turns failures into plain words: wrong login, blocked port or wrong URL, or unknown host. It also warns when the page is running inside the claude.ai viewer, which blocks outside connections.
- **Settings:** "Remember on this device" saves the address, username, password and site to localStorage key `shm-dash-conn`. Theme is saved to `shm-theme`. The command token is never stored.
- **Demo** (`05-app.js`, starts when nothing is saved):
- A week of captures every 5 minutes, with modes at 4.2, 13.3 and 16.3 Hz, shifting −0.09 %/°C with a daily temperature cycle.
- The first 12 hours are in learning ("collect") mode.
- Occasional false warnings; node n3 goes silent for 2 hours on day 4.4; a 4G outage on day 2.6.
- The last 80 captures have previews made from a modal simulation: 3 damped resonators with exact discretisation, vehicle crossings by time of day, noise, impacts and bad reads.
- A new capture arrives every 4 s.
- "Simulate 6% stiffness loss" drives the real 3-of-5 alarm logic.
- Demo commands work, including `send_raw`, which builds a real `.npz` in the browser so the same reader is exercised.

6. Work done, in order

1. **Built the full dashboard** to the brief: 6 pages, demo mode, MQTT, SQLite, `.npz` reader, light and dark themes, responsive layout, reduced-motion support.
2. **Wrote** `broker_test.py` (pure Python plus paho-mqtt; supports paho 1.x and 2.x). It has TLS on port 8883, a last will, retained state and online messages, a capture and preview every 10 s (every 3 s in live view), and replies to ping, status, live_view, live_stop and send_raw. Wrong tokens get `ok: false`. `--damage` forces an alarm; `--dry-run DIR` writes sample files without a broker.
3. **Tested everything in headless Chrome** with no console errors:
4. The alarm appears about 12 s after "Simulate 6% stiffness loss".
5. Every page, at 1366 px and at 390 px in dark mode with reduced motion.
6. A preview, capture and int16 `.npz` from Python decode to exactly the same rms and peak.
7. A test `remote.sqlite` loads.

8. **A live end-to-end run** on the public broker `broker.emqx.io`: the fake Pi published, the dashboard connected over wss on port 8084, the wrong token was rejected, ping got "pong", live view started, and a misspelled host got the "wrong address" message.
9. **UI/UX polish pass:**
10. Text contrast raised to at least 4.5:1 in both themes.
11. A skip link was added, focus moves to the main content when you change page, and the score chart has a screen-reader summary.
12. Touch targets are at least 44 px on touch screens, and buttons respond on hover and press.
13. Connect shows a loading state, the headline enters once on load, and the sticky header respects the safe area.
14. **Published the demo** to the private claude.ai page above.
15. **Created the local git repo**, renamed the output to `index.html` (for GitHub Pages), and wrote the README. The first commit is `6585a35`.

7. Known limitations and open items

- **Not yet tested against the owner's real Raspberry Pi.** The Pi's sender must match section 4 exactly; `broker_test.py` is the reference to compare against.
- **Heartbeats are not retained,** so right after connecting the System page may say "no heartbeat yet" for up to 15 minutes. "Request status" asks for one immediately.
- **Browsers don't say why a WebSocket failed,** so "wrong address" versus "blocked port" is a guess from timing: a fast failure means wrong address, a 16 s silence means a blocked port or wrong URL. Wrong login is reported by the broker itself.
- **Previews from `broker_test.py` are 13 to 14 kB,** not about 9 kB, because it encodes the waves at higher precision. The dashboard reads either.
- **The password is stored in plain text** in localStorage when "Remember on this device" is ticked. That was the brief's choice.
- **Inside the claude.ai viewer** only the demo and file pickers work; live MQTT does not.
- **Not done (possible next steps):** legends that toggle series on click; sortable captures table; CSV export; automated unit tests for `parseNpy` and `unzip`; a real laptop receiver script that writes `remote.sqlite` (none exists in this repo); a dark-mode screenshot review of every page.

8. How to change and test

1. Edit the files in `src\`, never `index.html`.
2. Rebuild by running `sh src/build.sh` in Git Bash.
3. Syntax check: `cat src/02-core.js src/03-overview.js src/04-pages.js src/05-app.js > all.js && node --check all.js`.
4. Browser test: `node tools/cdp-test.mjs tools/example-steps.json`. It prints the results of the in-page JS and any console errors, and saves screenshots. Step options are `size` [w,h], `dark`, `rm` (reduced motion), `nav` (URL), `wait`, `eval` (JS run in the page; globals such as `S`, `R`, `DEMO`, `toggleDamage()` and `sendCmd()` are reachable), `sleep`, `shot` (file) and `maxH`. Chrome is expected at `C:/Program Files/Google/Chrome/Application/chrome.exe`.
5. Fake Pi without a broker: `python broker_test.py --dry-run out`.

6. Fake Pi with a broker: `python broker_test.py --host <addr> --user <pi user> --password <pi pass> --token <token>`. For a quick free test use `--host broker.emqx.io` (no login needed) with a unique `--site`, and connect the dashboard to `broker.emqx.io` with the same site ID.

9. Uploading to GitHub (owner's step, not done yet)

1. On github.com, signed in as `praveenregar30`, create a new repository named `spanguard` with no README, .gitignore or license.
2. In PowerShell: `cd C:\Users\77263\bridge-shm git remote add origin https://github.com/praveenregar30/spanguard.git git push -u origin main`
3. Optional: Settings, then Pages, then Deploy from a branch, `main`, `/ (root)`. The site appears at `https://praveenregar30.github.io/spanguard/` (needs a public repository on a free account).
4. Later changes: `git add -A`, `git commit -m "..."`, `git push`.

Note: commits on this laptop use the email `praveenregar32@gmail.com`. Add it to the GitHub account (Settings, Emails) so commits link to the profile.